Data Science Lab Practicals — All Programs (Copy-Paste Ready)

IMPORTANT: Install these libraries first (run once in terminal or Colab cell):

> `pip install numpy pandas matplotlib seaborn scikit-learn mlxtend openpyxl` Every program below is **tested and verified**. Copy one program at a time into a Jupyter/Colab cell and run it. LA 13 box plot has been **corrected** (`labels` --> `tick_labels` for matplotlib >= 3.9).

LA 1 — Program 1: Combine three vectors into a 3x3 matrix

Aim: Create three vectors and make each vector a column of the matrix.

```
import numpy as np

a = [1, 2, 3]
b = [4, 5, 6]
c = [7, 8, 9]

matrix = np.column_stack((a, b, c))

print("3 x 3 matrix:")
print(matrix)
```

Expected Output:

```
3 x 3 matrix:
[[1 4 7]
 [2 5 8]
 [3 6 9]]
```

LA 1 — Program 2: List containing a vector, matrix and list

Aim: Name the elements and access the first and second elements.

```
import numpy as np

my_list = {
    "vector": [1, 2, 3],
    "matrix": np.array([[1, 2], [3, 4]]),
    "list": ["Apple", "Banana", "Mango"]
}

print("First element (vector):", my_list["vector"])
print("Second element (matrix):")
print(my_list["matrix"])
```

Expected Output:

```
First element (vector): [1, 2, 3]
Second element (matrix):
[[1 2]
 [3 4]]
```

LA 1 — Program 3: Three-dimensional array with two 3x3 tables

Aim: Create an array with three rows, three columns and two tables using two vectors.

```
import numpy as np

# Each input vector contains 9 values; together they form 18 values.
a = list(map(int, input("Enter 9 integers for vector a: ").split()))
b = list(map(int, input("Enter 9 integers for vector b: ").split()))

if len(a) != 9 or len(b) != 9:
    raise ValueError("Enter exactly 9 integers for each vector.")

arr = np.array(a + b).reshape(2, 3, 3)

print("Array with 2 tables, each 3 x 3:")
print(arr)
```

Example Input:

```
Enter 9 integers for vector a: 1 2 3 4 5 6 7 8 9
Enter 9 integers for vector b: 10 11 12 13 14 15 16 17 18
```

Expected Output:

```
Array with 2 tables, each 3 x 3:
[[[ 1  2  3]
  [ 4  5  6]
  [ 7  8  9]]

 [[10 11 12]
  [13 14 15]
  [16 17 18]]]
```

LA 1 — Program 4: Create a DataFrame from four vectors

Aim: Create a student DataFrame containing name, score, attempts and qualification.

```
import pandas as pd

name = ["Anastasia", "Dima", "Katherine", "James", "Emily",
        "Michael", "Matthew", "Laura", "Kevin", "Jonas"]
score = [12.5, 9, 16.5, 12, 9, 20, 14.5, 13.5, 8, 19]
attempts = [1, 3, 2, 3, 2, 3, 1, 1, 2, 1]
qualify = ["yes", "no", "yes", "no", "no",
           "yes", "yes", "no", "no", "yes"]

df = pd.DataFrame({
    "name": name,
    "score": score,
    "attempts": attempts,
    "qualify": qualify
})

print(df)
```

Expected Output:

```
        name  score  attempts qualify
0  Anastasia   12.5         1     yes
1       Dima    9.0         3      no
2  Katherine   16.5         2     yes
3      James   12.0         3      no
4      Emily    9.0         2      no
5    Michael   20.0         3     yes
6    Matthew   14.5         1     yes
7      Laura   13.5         1      no
8      Kevin    8.0         2      no
9      Jonas   19.0         1     yes
```

LA 1 — Program 5: Create a categorical factor for women's height

Aim: Represent the classic R women dataset height values as a categorical variable in Python.

```
import pandas as pd

# Classic R women dataset uses heights from 58 to 72 inches.
height = list(range(58, 73))

height_factor = pd.Categorical(height)

print("Height factor:")
print(height_factor)
print("Categories:", list(height_factor.categories))
```

Expected Output:

```
Height factor:
[58, 59, 60, 61, 62, ..., 68, 69, 70, 71, 72]
Length: 15
Categories (15, int64): [58, 59, 60, 61, ..., 69, 70, 71, 72]
Categories: [58, 59, 60, 61, 62, 63, 64, 65, 66, 67, 68, 69, 70, 71, 72]
```

LA 1 — Program 6: Print the Fibonacci sequence

Aim: Print the first n terms of the Fibonacci sequence.

```
n = int(input("Enter number of terms: "))

a, b = 0, 1

for i in range(n):
    print(a, end=" ")
    a, b = b, a + b

print()
```

Example Input:

```
Enter number of terms: 10
```

Expected Output:

```
0 1 1 2 3 5 8 13 21 34
```

LA 1 — Program 7: Export and import CSV, Excel and TXT files

Aim: Create sample data, export it to three file formats, and read the files back.

```
import pandas as pd

df = pd.DataFrame({
    "Name": ["Amit", "Neha", "Raj"],
    "Marks": [80, 90, 75]
})

# Export
df.to_csv("data.csv", index=False)
df.to_excel("data.xlsx", index=False)
df.to_csv("data.txt", sep="\t", index=False)

# Import
print("CSV file:")
print(pd.read_csv("data.csv"))

print("\nExcel file:")
print(pd.read_excel("data.xlsx"))

print("\nTXT file:")
print(pd.read_csv("data.txt", sep="\t"))
```

Expected Output:

```
CSV file:
   Name  Marks
0  Amit     80
1  Neha     90
2   Raj     75

Excel file:
   Name  Marks
0  Amit     80
1  Neha     90
2   Raj     75

TXT file:
   Name  Marks
0  Amit     80
1  Neha     90
2   Raj     75
```

LA 1 — Program 8: Reduce a number to one digit by repeatedly summing digits

Aim: Use loops to calculate the digital root of a non-negative integer.

```
n = abs(int(input("Enter an integer: ")))

while n >= 10:
    total = 0
    while n > 0:
        total += n % 10
        n //= 10
    n = total

print("Single digit:", n)
```

Example Input:

```
Enter an integer: 9875
```

Expected Output:

```
Single digit: 2
```

LA 2: Iris classification using Logistic Regression

Aim: Classify Iris-setosa versus all other species and calculate accuracy, precision and recall.

```
from sklearn.datasets import load_iris
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import accuracy_score, precision_score, recall_score

iris = load_iris()
X = iris.data
y = (iris.target == 0).astype(int)  # 1 = Iris-setosa; 0 = other species

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42, stratify=y
)

model = LogisticRegression(max_iter=500)
model.fit(X_train, y_train)
y_pred = model.predict(X_test)

print("Accuracy:", accuracy_score(y_test, y_pred))
print("Precision:", precision_score(y_test, y_pred, zero_division=0))
print("Recall:", recall_score(y_test, y_pred, zero_division=0))
```

Expected Output:

```
Accuracy: 1.0
Precision: 1.0
Recall: 1.0
```

LA 3: Titanic survival prediction using a Decision Tree

Aim: Load Titanic CSV data, handle missing values, encode categories, train a Decision Tree and plot its confusion matrix.

```
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
from sklearn.model_selection import train_test_split
from sklearn.tree import DecisionTreeClassifier
from sklearn.metrics import confusion_matrix, accuracy_score

url = "https://raw.githubusercontent.com/datasciencedojo/datasets/master/titanic.csv"

try:
    df = pd.read_csv(url)  # Titanic CSV from public URL
except Exception:
    # Small fallback dataset so the program still runs without internet.
    df = pd.DataFrame({
        "Pclass": [1, 3, 2, 1, 3, 2, 3, 1, 2, 3, 1, 3],
        "Sex": ["female", "male", "female", "male", "male", "female",
                "male", "female", "male", "female", "male", "male"],
        "Age": [29, 22, 35, 40, 18, 27, 31, 24, 45, 19, 50, 28],
        "SibSp": [0, 1, 0, 0, 0, 1, 0, 0, 0, 0, 1, 0],
        "Parch": [0, 0, 0, 0, 0, 0, 0, 1, 0, 0, 0, 0],
        "Fare": [80, 7, 20, 50, 8, 15, 9, 70, 12, 8, 40, 7],
        "Embarked": ["S", "S", "C", "S", "Q", "S", "S", "C", "S", "Q", "C", "S"],
        "Survived": [1, 0, 1, 0, 0, 1, 0, 1, 0, 1, 0, 0]
    })
    print("Internet unavailable: using a small built-in demonstration dataset.")

features = ["Pclass", "Sex", "Age", "SibSp", "Parch", "Fare", "Embarked"]
X = df[features].copy()
y = df["Survived"]

X["Age"] = X["Age"].fillna(X["Age"].median())
X["Fare"] = X["Fare"].fillna(X["Fare"].median())
X["Embarked"] = X["Embarked"].fillna(X["Embarked"].mode()[0])

X = pd.get_dummies(X, columns=["Sex", "Embarked"], drop_first=True)

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42, stratify=y
)

model = DecisionTreeClassifier(max_depth=4, random_state=42)
model.fit(X_train, y_train)
y_pred = model.predict(X_test)

print("Accuracy:", accuracy_score(y_test, y_pred))

cm = confusion_matrix(y_test, y_pred)
sns.heatmap(cm, annot=True, fmt="d", cmap="Blues")
plt.xlabel("Predicted")
plt.ylabel("Actual")
plt.title("Titanic Confusion Matrix")
plt.tight_layout()
plt.show()
```

Expected Output:

```
Accuracy: ~0.78 (varies with dataset)
+ A confusion matrix heatmap is displayed
```

LA 4: Movie review sentiment analysis using Naive Bayes

Aim: Classify movie reviews as positive or negative based on review text.

```
from sklearn.feature_extraction.text import CountVectorizer
from sklearn.naive_bayes import MultinomialNB
from sklearn.pipeline import make_pipeline

reviews = [
    "This movie is excellent and amazing",
    "I loved this wonderful film",
    "The movie was fantastic",
    "A brilliant and enjoyable movie",
    "This film is boring and terrible",
    "I hated this awful movie",
    "The movie was very bad",
    "A disappointing and dull film"
]

labels = [
    "positive", "positive", "positive", "positive",
    "negative", "negative", "negative", "negative"
]

model = make_pipeline(CountVectorizer(), MultinomialNB())
model.fit(reviews, labels)

new_reviews = [
    "The movie was amazing and wonderful",
    "The film was dull and terrible"
]

predictions = model.predict(new_reviews)

for review, sentiment in zip(new_reviews, predictions):
    print("Review:", review)
    print("Sentiment:", sentiment)
    print()
```

Expected Output:

```
Review: The movie was amazing and wonderful
Sentiment: positive

Review: The film was dull and terrible
Sentiment: negative
```

LA 5: Salary prediction using Simple Linear Regression

Aim: Model salary as a function of years of experience using 100 example employee records.

```
import numpy as np
import matplotlib.pyplot as plt
from sklearn.linear_model import LinearRegression

# Synthetic data for 100 employees (reproducible example)
rng = np.random.default_rng(42)
experience = np.linspace(0, 20, 100)
salary = 30000 + 4500 * experience + rng.normal(0, 5000, 100)

X = experience.reshape(-1, 1)
model = LinearRegression()
model.fit(X, salary)

years = 6.5
predicted_salary = model.predict([[years]])[0]

print("Slope:", model.coef_[0])
print("Intercept:", model.intercept_)
print(f"Predicted salary for {years} years of experience: {predicted_salary:.2f}")

plt.scatter(experience, salary, label="Employee data")
plt.plot(experience, model.predict(X), label="Regression line")
plt.xlabel("Years of Experience")
plt.ylabel("Salary")
plt.title("Salary Prediction")
plt.legend()
plt.tight_layout()
plt.show()
```

Expected Output:

```
Slope: 4395.117908240689
Intercept: 30797.472860173817
Predicted salary for 6.5 years of experience: 59365.74
+ A scatter plot with regression line is displayed
```

LA 6: House price prediction using Linear Regression

Aim: Predict house price from area, plot the regression line, and calculate MSE and R^2.

```
import numpy as np
import matplotlib.pyplot as plt
from sklearn.linear_model import LinearRegression
from sklearn.model_selection import train_test_split
from sklearn.metrics import mean_squared_error, r2_score

# Sample area in square feet and price in sample units
area = np.array([500, 700, 800, 1000, 1200, 1400, 1600, 1800, 2000, 2200])
price = np.array([15, 20, 22, 28, 32, 38, 42, 48, 52, 58])

X = area.reshape(-1, 1)
X_train, X_test, y_train, y_test = train_test_split(
    X, price, test_size=0.2, random_state=42
)

model = LinearRegression()
model.fit(X_train, y_train)
y_pred = model.predict(X_test)

print("Predictions:", y_pred)
print("MSE:", mean_squared_error(y_test, y_pred))
print("R2 score:", r2_score(y_test, y_pred))

order = np.argsort(area)
plt.scatter(area, price, label="Actual data")
plt.plot(area[order], model.predict(X[order]), label="Regression line")
plt.xlabel("Area (square feet)")
plt.ylabel("House price (sample units)")
plt.title("House Price Prediction")
plt.legend()
plt.tight_layout()
plt.show()
```

Expected Output:

```
Predictions: [52.80046539 19.85049447]
MSE: 0.3315483689970853
R2 score: 0.9987048891836051
+ A scatter plot with regression line is displayed
```

LA 7: K-Means clustering with K=7 on tissue-like gene-expression data

Aim: Run K-Means with seven clusters, compare clusters with known example labels, and repeat with different initializations.

```
import pandas as pd
from sklearn.datasets import make_blobs
from sklearn.preprocessing import StandardScaler
from sklearn.cluster import KMeans
from sklearn.metrics import adjusted_rand_score

# Self-contained synthetic substitute for tissue_gene_expression.csv
# 7 simulated tissue types, 20 gene-expression features
X, actual_tissue = make_blobs(
    n_samples=210, centers=7, n_features=20,
    cluster_std=2.0, random_state=42
)

tissue_names = [f"Tissue_{i + 1}" for i in actual_tissue]
X_scaled = StandardScaler().fit_transform(X)

for seed in [0, 10, 20, 30, 40]:
    model = KMeans(n_clusters=7, random_state=seed, n_init=10)
    clusters = model.fit_predict(X_scaled)

    table = pd.crosstab(
        pd.Series(clusters, name="K-Means Cluster"),
        pd.Series(tissue_names, name="Actual Tissue Type")
    )

    print("\nRandom state:", seed)
    print(table)
    print("Adjusted Rand Index:", round(
        adjusted_rand_score(actual_tissue, clusters), 4
    ))
```

Expected Output:

```
Random state: 0
(crosstab table showing perfect cluster assignment)
Adjusted Rand Index: 1.0

Random state: 10
(crosstab table)
Adjusted Rand Index: 1.0
... (repeats for seeds 20, 30, 40 — all ARI = 1.0)
```

LA 8: Build an FP-Tree and show its evolution

Aim: Use a minimum support of 0.001 and print the prefix tree after each transaction.

```
from collections import Counter
from math import ceil

transactions = [
    ["Milk", "Bread", "Butter"],
    ["Bread", "Butter"],
    ["Milk", "Bread"],
    ["Milk", "Butter"],
    ["Bread", "Butter", "Eggs"]
]

min_support = 0.001
min_count = ceil(min_support * len(transactions))

frequency = Counter(item for transaction in transactions for item in transaction)
frequent_items = {
    item for item, count in frequency.items() if count >= min_count
}

root = {"count": 0, "children": {}}

def insert_transaction(items):
    node = root
    node["count"] += 1
    for item in items:
        if item not in node["children"]:
            node["children"][item] = {"count": 0, "children": {}}
        node = node["children"][item]
        node["count"] += 1

def print_tree(node, indent=""):
    for item, child in node["children"].items():
        print(f"{indent}{item} ({child['count']})")
        print_tree(child, indent + "  ")

print("Item frequencies:", dict(frequency))
print("Minimum support count:", min_count)

for number, transaction in enumerate(transactions, start=1):
    ordered = sorted(
        [item for item in transaction if item in frequent_items],
        key=lambda item: (-frequency[item], item)
    )
    insert_transaction(ordered)
    print(f"\nAfter transaction {number}: {ordered}")
    print_tree(root)
```

Expected Output:

```
Item frequencies: {'Milk': 3, 'Bread': 4, 'Butter': 4, 'Eggs': 1}
Minimum support count: 1

After transaction 1: ['Bread', 'Butter', 'Milk']
Bread (1)
  Butter (1)
    Milk (1)

After transaction 2: ['Bread', 'Butter']
Bread (2)
  Butter (2)
    Milk (1)

After transaction 3: ['Bread', 'Milk']
Bread (3)
  Butter (2)
    Milk (1)
  Milk (1)

After transaction 4: ['Butter', 'Milk']
Bread (3)
  Butter (2)
    Milk (1)
  Milk (1)
Butter (1)
  Milk (1)

After transaction 5: ['Bread', 'Butter', 'Eggs']
Bread (4)
  Butter (3)
    Milk (1)
    Eggs (1)
  Milk (1)
Butter (1)
  Milk (1)
```

LA 9: Apriori — top five association rules

Aim: Use minimum support 0.001 and confidence 0.8, then sort rules by confidence.

```
import pandas as pd
from mlxtend.preprocessing import TransactionEncoder
from mlxtend.frequent_patterns import apriori, association_rules

transactions = [
    ["Milk", "Bread", "Butter"],
    ["Bread", "Butter"],
    ["Milk", "Bread"],
    ["Milk", "Butter"],
    ["Bread", "Butter", "Eggs"],
    ["Milk", "Bread", "Butter"],
    ["Milk", "Bread", "Butter"]
]

encoder = TransactionEncoder()
encoded = encoder.fit(transactions).transform(transactions)
df = pd.DataFrame(encoded, columns=encoder.columns_)

frequent_itemsets = apriori(
    df, min_support=0.001, use_colnames=True
)

rules = association_rules(
    frequent_itemsets, metric="confidence", min_threshold=0.8
).sort_values("confidence", ascending=False)

if rules.empty:
    print("No rules meet the confidence threshold.")
else:
    display_rules = rules.copy()
    display_rules["antecedents"] = display_rules["antecedents"].apply(
        lambda s: ", ".join(sorted(s))
    )
    display_rules["consequents"] = display_rules["consequents"].apply(
        lambda s: ", ".join(sorted(s))
    )
    print("Top 5 rules sorted by confidence:")
    print(display_rules[
        ["antecedents", "consequents", "support", "confidence", "lift"]
    ].head(5).to_string(index=False))

    strong = rules[(rules["confidence"] >= 0.8) & (rules["lift"] > 1)]
    print("\nStrong rules (confidence >= 0.8 and lift > 1):")
    print(len(strong), "strong rule(s) found.")
```

Expected Output:

```
Top 5 rules sorted by confidence:
 antecedents   consequents  support  confidence     lift
        Eggs         Bread 0.142857         1.0 1.166667
Butter, Eggs         Bread 0.142857         1.0 1.166667
        Eggs        Butter 0.142857         1.0 1.166667
 Bread, Eggs        Butter 0.142857         1.0 1.166667
        Eggs Bread, Butter 0.142857         1.0 1.400000

Strong rules (confidence >= 0.8 and lift > 1):
5 strong rule(s) found.
```

LA 10: FP-Growth for Market Basket Analysis

Aim: Find frequent itemsets and association rules using the FP-Growth algorithm.

```
import pandas as pd
from mlxtend.preprocessing import TransactionEncoder
from mlxtend.frequent_patterns import fpgrowth, association_rules

transactions = [
    ["Milk", "Bread", "Butter"],
    ["Bread", "Butter"],
    ["Milk", "Bread"],
    ["Milk", "Butter"],
    ["Bread", "Butter", "Eggs"],
    ["Milk", "Bread", "Butter"],
    ["Milk", "Bread", "Butter"]
]

encoder = TransactionEncoder()
encoded = encoder.fit(transactions).transform(transactions)
df = pd.DataFrame(encoded, columns=encoder.columns_)

frequent_itemsets = fpgrowth(
    df, min_support=0.001, use_colnames=True
)

print("Frequent itemsets:")
print(frequent_itemsets.to_string(index=False))

rules = association_rules(
    frequent_itemsets, metric="confidence", min_threshold=0.8
).sort_values("confidence", ascending=False)

print("\nAssociation rules:")
if rules.empty:
    print("No rules meet the confidence threshold.")
else:
    display_rules = rules.copy()
    display_rules["antecedents"] = display_rules["antecedents"].apply(
        lambda s: ", ".join(sorted(s))
    )
    display_rules["consequents"] = display_rules["consequents"].apply(
        lambda s: ", ".join(sorted(s))
    )
    print(display_rules[
        ["antecedents", "consequents", "support", "confidence", "lift"]
    ].to_string(index=False))
```

Expected Output:

```
Frequent itemsets:
 support                        itemsets
0.857143               frozenset({Bread})
0.857143              frozenset({Butter})
0.714286                frozenset({Milk})
0.142857                frozenset({Eggs})
0.714286       frozenset({Bread, Butter})
...

Association rules:
 antecedents   consequents  support  confidence     lift
        Eggs         Butter 0.142857    1.000000 1.166667
 Bread, Eggs         Butter 0.142857    1.000000 1.166667
        Eggs          Bread 0.142857    1.000000 1.166667
...
```

LA 11: Stacked bar chart of monthly revenue by region

Aim: Plot months on the X-axis, revenue on the Y-axis, and add a legend for East, West and North.

```
import numpy as np
import matplotlib.pyplot as plt

months = ["Mar", "Apr", "May", "Jun", "Jul"]
regions = ["East", "West", "North"]
colors = ["green", "orange", "brown"]

east = np.array([20, 25, 30, 35, 40])
west = np.array([15, 20, 25, 30, 35])
north = np.array([10, 15, 20, 25, 30])

plt.bar(months, east, color=colors[0], label=regions[0])
plt.bar(months, west, bottom=east, color=colors[1], label=regions[1])
plt.bar(months, north, bottom=east + west, color=colors[2], label=regions[2])

plt.xlabel("Month")
plt.ylabel("Revenue")
plt.title("Monthly Revenue by Region")
plt.legend()
plt.tight_layout()
plt.show()
```

Expected Output:

```
A stacked bar chart with three colored segments per month is displayed.
```

LA 12: Bar chart of critics_score for the first ten movies

Aim: Read moviesData.csv, plot critics_score for the first ten movies, and save the chart.

```
from pathlib import Path
import pandas as pd
import matplotlib.pyplot as plt

file_path = Path("moviesData.csv")

if file_path.exists():
    df = pd.read_csv(file_path)
    if "critics_score" not in df.columns:
        raise ValueError(
            "The CSV must contain a column named 'critics_score'. "
            f"Available columns: {list(df.columns)}"
        )
    data = df.head(10).copy()
    if "movie_title" in data.columns:
        labels = data["movie_title"].astype(str)
    elif "title" in data.columns:
        labels = data["title"].astype(str)
    else:
        labels = data.index.astype(str)
else:
    # Fallback data allows the chart code to run without the file.
    print("moviesData.csv not found; using sample scores for demonstration.")
    data = pd.DataFrame({
        "movie_title": [
            "Movie 1", "Movie 2", "Movie 3", "Movie 4", "Movie 5",
            "Movie 6", "Movie 7", "Movie 8", "Movie 9", "Movie 10"
        ],
        "critics_score": [78, 85, 62, 91, 73, 88, 69, 95, 80, 76]
    })
    labels = data["movie_title"]

plt.figure(figsize=(10, 5))
plt.bar(labels, data["critics_score"])
plt.xlabel("Movie")
plt.ylabel("Critics Score")
plt.title("Critics' Scores for First 10 Movies")
plt.xticks(rotation=45, ha="right")
plt.tight_layout()
plt.savefig("critics_score.png", dpi=300)
plt.show()

print("Chart saved as critics_score.png")
```

Expected Output:

```
moviesData.csv not found; using sample scores for demonstration.
Chart saved as critics_score.png
+ A bar chart of critics scores is displayed
```

LA 13: Box plots for mpg and cyl from mtcars [CORRECTED] CORRECTED

Aim: Create box plots for the mpg and cyl columns using values embedded directly in the program.

WARNING: Original error: `labels=` parameter was removed in matplotlib >= 3.9. The corrected code below uses `tick_labels=` instead.

```
import pandas as pd
import matplotlib.pyplot as plt

# mpg and cyl values from the classic mtcars dataset
df = pd.DataFrame({
    "mpg": [
        21.0, 21.0, 22.8, 21.4, 18.7, 18.1, 14.3, 24.4,
        22.8, 19.2, 17.8, 16.4, 17.3, 15.2, 10.4, 10.4,
        14.7, 32.4, 30.4, 33.9, 21.5, 15.5, 15.2, 13.3,
        19.2, 27.3, 26.0, 30.4, 15.8, 19.7, 15.0, 21.4
    ],
    "cyl": [
        6, 6, 4, 6, 8, 6, 8, 4, 4, 6, 6, 8, 8, 8, 8, 8,
        8, 4, 4, 4, 4, 8, 8, 8, 8, 4, 4, 4, 8, 6, 8, 4
    ]
})

print(df.head())

plt.figure(figsize=(7, 5))
plt.boxplot([df["mpg"], df["cyl"]], tick_labels=["MPG", "Cylinders"])
plt.title("Box Plot of MPG and Cylinders")
plt.ylabel("Value")
plt.tight_layout()
plt.show()
```

Expected Output:

```
    mpg  cyl
0  21.0    6
1  21.0    6
2  22.8    4
3  21.4    6
4  18.7    8
+ A box plot comparing MPG and Cylinders is displayed
```

Quick Viva Q&A (30 Questions)

Python Basics (Programs 1–8)

Q1: What does `np.column_stack()` do? A: It takes multiple 1D arrays or lists and stacks them side by side as columns to form a 2D matrix.

Q2: Is `my_list` in Program 2 actually a list? A: No, it is a Python dictionary (`dict`). It uses key-value pairs to mimic R's named list. A Python list uses numeric indexing `[0], [1]`, while a dict uses named keys like `["vector"]`.

Q3: What does `reshape(2, 3, 3)` mean? A: It converts a flat 1D array into a 3D array — 2 tables, each with 3 rows and 3 columns. Total elements must equal 2 x 3 x 3 = 18.

Q4: What is `pd.Categorical()`? A: The Python/pandas equivalent of R's `factor()`. It marks data as having a fixed set of allowed values (categories).

Q5: What is a digital root? A: You keep summing the digits of a number until you get a single digit. Example: 9875 --> 29 --> 11 --> 2.

Q6: Why do we need `openpyxl`? A: pandas uses `openpyxl` as the engine to read and write `.xlsx` (Excel) files. Without it, `to_excel()` and `read_excel()` will throw an error.

Q7: What is the difference between CSV and Excel format? A: CSV is plain text with comma-separated values. Excel (`.xlsx`) is a binary format that supports formatting, formulas, and multiple sheets.

Q8: What is the Fibonacci sequence? A: A sequence where each number is the sum of the two before it: 0, 1, 1, 2, 3, 5, 8, 13, 21, …

Classification (LA 2, 3, 4)

Q9: What is Logistic Regression? A: A classification algorithm that uses a sigmoid function to predict the probability of a binary outcome (0 or 1).

Q10: Why is accuracy 100% for Iris-setosa? A: Because Iris-setosa is linearly separable from the other species — its petal and sepal measurements are distinctly different.

Q11: What is the difference between accuracy, precision, and recall? A: **Accuracy** = correct / total. **Precision** = true positives / predicted positives ("how many of my 'yes' predictions were correct?"). **Recall** = true positives / actual positives ("how many actual positives did I catch?").

Q12: What is a Decision Tree? A: A model that splits data into branches based on feature conditions (like a flowchart). Each leaf node gives a prediction.

Q13: What is a confusion matrix? A: A table that shows True Positives, True Negatives, False Positives, and False Negatives — comparing actual vs predicted labels.

Q14: Why is Naive Bayes called "naive"? A: Because it assumes all features are independent of each other. In text classification, it assumes each word is independent of every other word.

Q15: What is `CountVectorizer`? A: It converts text into a matrix of word counts (bag-of-words). Each row is a document, each column is a unique word, and the value is how many times that word appears.

Q16: What does `make_pipeline()` do? A: It chains multiple steps (e.g., vectorizer + classifier) into a single object. You call `.fit()` and `.predict()` once and it runs all steps in order.

Regression (LA 5, 6)

Q17: What is Linear Regression? A: A model that fits a straight line (y = mx + b) through data to predict a continuous numeric output.

Q18: What do slope and intercept mean? A: **Slope** = how much the output changes per 1-unit increase in input. **Intercept** = the predicted output when input is zero.

Q19: What is MSE (Mean Squared Error)? A: The average of squared differences between actual and predicted values. Lower MSE = better model.

Q20: What is R^2 score? A: It measures how much of the variation in the target is explained by the model. R^2 = 1 means perfect fit. R^2 = 0 means the model is no better than predicting the average.

Q21: Why is R^2 so high (0.999) in LA 6? A: Because the sample data has an almost perfectly linear relationship. In real datasets, R^2 of 0.7–0.9 is considered good.

Clustering (LA 7)

Q22: What is K-Means clustering? A: An unsupervised algorithm that groups data into K clusters by minimizing the distance between each point and its nearest cluster center (centroid).

Q23: What is Adjusted Rand Index (ARI)? A: A score measuring agreement between two clusterings. ARI = 1 means perfect match; ARI ~ 0 means random.

Q24: Why use `StandardScaler`? A: K-Means uses distance. If features have different scales (e.g., age 0–100 vs salary 0–100000), the larger-scale feature will dominate. Scaling makes all features equally important.

Q25: What does `n_init=10` mean? A: K-Means runs 10 times with different random starting points and picks the best result, reducing the chance of a bad initialization.

Association Rules (LA 8, 9, 10)

Q26: What is an FP-Tree? A: A Frequent Pattern Tree that compresses transaction data into a prefix tree, allowing efficient mining of frequent patterns without generating candidates.

Q27: What is the difference between Apriori and FP-Growth? A: **Apriori** generates candidate itemsets level by level and scans the database multiple times. **FP-Growth** builds an FP-Tree and mines patterns directly — faster and needs only 2 database scans.

Q28: What are Support, Confidence, and Lift? A: **Support** = fraction of transactions containing the itemset. **Confidence** of A-->B = support(A U B) / support(A). **Lift** = confidence / support(B). Lift > 1 means positive association.

Visualization (LA 11, 12, 13)

Q29: How does a stacked bar chart work? A: Each bar is split into colored segments. The `bottom` parameter in `plt.bar()` stacks each region on top of the previous one.

Q30: What does a box plot show? A: It shows the **median** (center line), **Q1 and Q3** (box edges), **whiskers** (data range), and **outliers** (dots beyond whiskers). It summarizes the spread and skewness of data.